Closet Catalog & Outfit Builder - Project 4

Fatima Qasem (Frontend, Database, Mockup), Sana Syed (Backend, Database, Tutorial)

University of Michigan Dearborn

CIS 435

Prof. John Baugh

07 Dec 2025

Design Considerations

Our project implements a client-server architecture with three distinct layers. The frontend is a Single Page Application built with HTML, CSS, and JavaScript that handles the user interface and browser interactions. The backend, built with Node.js, manages data operations and business logic while communicating with the database. The MySQL database provides persistent storage through structured tables and relationships. This clear separation allows each layer to function independently, making the system maintainable, testable, and scalable by design. Issues in one layer don't affect others, and components can be updated or scaled separate

A responsive grid layout forms the foundation of the frontend design. A blurred background image with a warm overlay and subtle glass-like effects that give cards a translucent look complement the interface's elegant champagne and rose gold color scheme, which is in line with fashion themes. Forms with a two-column input structure that neatly compresses on smaller devices are sensibly developed. To develop an easy-to-use interface, interactive elements include a checkbox pick list for putting together outfits, erase buttons, and manual refresh controls.

With queries to stop SQL injection, input sanitization through trimming and null handling, CORS settings for safe cross-origin communication, and transaction management for data integrity, the backend is built with security as its top priority. The database design is based on a relational model, with distinct tables for outfit definitions and clothing items linked by a join table. With logic endpoints for CRUD operations, suitable use of HTTP methods, and status codes that clearly convey request outcomes, the API complies with RESTful standards. Error handling creates a secure and well-organized foundation for the application by encasing business

logic in try-catch blocks, offering a user-friendly error response while logging technical information and guaranteeing transaction rollback on failures to prevent partial data corruption.

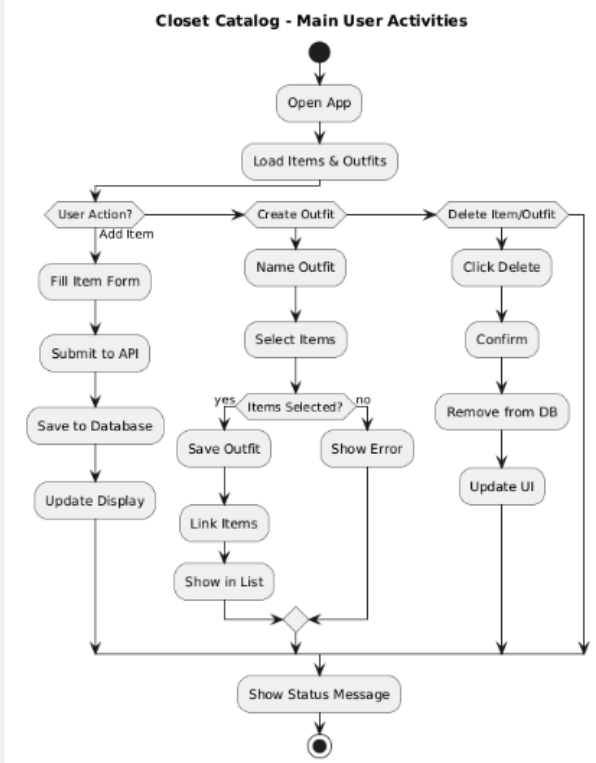
Database connection pooling, effective batch operations, frontend minimal DOM updates with template strings, and asset optimization with WebP format and CSS variables all improve performance. Progressive disclosure, which gradually discloses complexity, extensive feedback systems, such as loading states and status messages, and accessibility elements, such as semantic HTML and sufficient color contrast, are all incorporated into the design of the user experience. A cohesive system that strikes a balance between speed, usability, and developed ergonomics is created by code organization that adheres to modular principles with distinct route files and isolated database configuration, maintains readability through descriptive function names and consistent formatting, ensures maintainability with centralized API configuration, and uniform error handling patterns.

User Interface Design

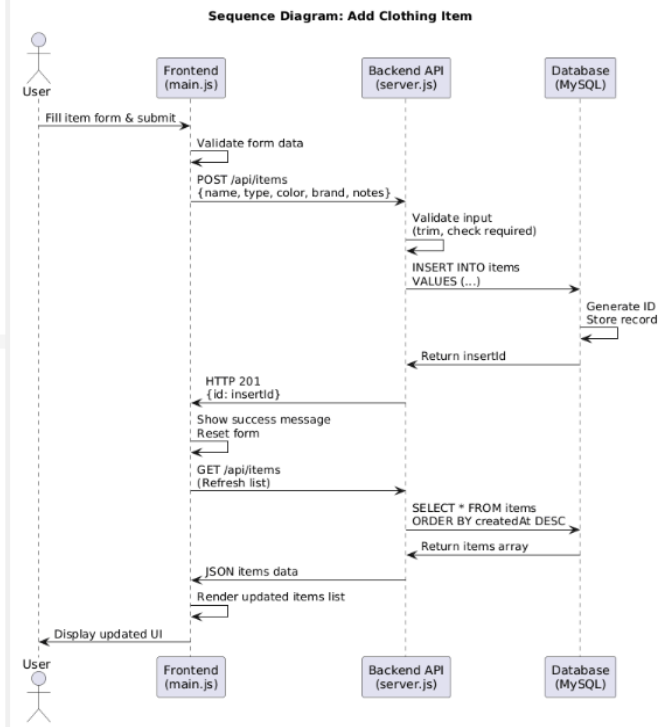
Mockup:

System Components and Functions

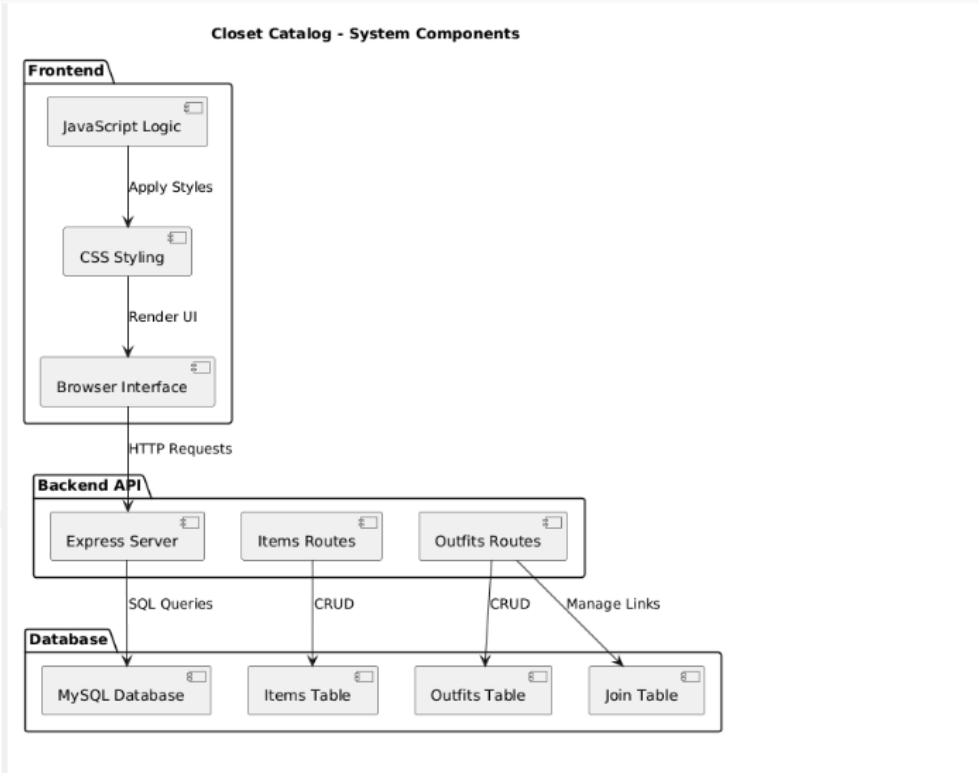
Activity Diagram:



Sequence Diagram:



Component Diagram:



Conclusion

We discovered how to create a full-stack application with distinct frontend, backend, and database layers while working on the Closet Catalog + Outfit Builder. We developed hands-on experience using Express.js to create RESTful APIs, designing relational database schemas with appropriate many-to-many relationships, and using contemporary CSS to create responsive user interfaces. The project reaffirmed the significance of transaction management for data integrity, appropriate error handling throughout the stack, and upholding a clear design with distinct roles for each component. We have developed a working wardrobe management system that shows how several technologies may cooperate to address practical issues while preserving maintainability and scalability.